Établissement : Tekup

filière : GLSI

Rapport de projet

Application de gestion et d'entraînement sportif

Réalisé par : Ahmed ikenjatoun
Année universitaire : 2025–2026

Résumé

La plateforme Fitness étudiée dans ce rapport associe des fonctions de préparation et de suivi de l'entraînement à un catalogue de programmes et à une boutique. Le projet repose sur une interface web Next.js et React, une API NestJS et une base PostgreSQL manipulée avec Prisma. L'analyse du code distingue les parcours accessibles par l'API, les écrans de démonstration et les services métier qui ne sont pas encore raccordés à des routes. Le rapport présente le contexte, les besoins, l'architecture observée et les principaux domaines fonctionnels, puis précise les limites établies à partir du dépôt.
Mots-clés : entraînement sportif, application web, Next.js, NestJS, PostgreSQL, Prisma, UML.

Abstract

This report studies a Fitness platform that combines workout planning and tracking with training programs and a product catalog. The project uses a Next.js and React web interface, a NestJS API, and PostgreSQL accessed through Prisma. The source review distinguishes API-backed user journeys from demonstration screens and service logic that is not yet connected to routes. The report describes the context, requirements, observed architecture, and main functional areas, while identifying limitations supported by the repository.

Keywords : fitness training, web application, Next.js, NestJS, PostgreSQL, Prisma, UML.

Table des matières

Liste des figures **v**

Liste des tableaux **vi**

Liste des abréviations **vii**

Introduction générale **1**

1 Cadre général du projet **2**

Introduction . . . 3

1.1 Contexte général . . . 3

1.2 Présentation du projet . . . 3

1.2.1 Cadre général . . . 3

1.2.2 Description du projet . . . 3

1.2.3 Étude de l'existant . . . 4

1.2.4 Problématique . . . 4

1.2.5 Solution proposée . . . 4

1.3 Présentation de la méthodologie de travail . . . 5

1.3.1 Formalisme de modélisation . . . 5

1.3.2 Méthodologie adoptée . . . 5

Conclusion . . . 7

2 Analyse et spécification des besoins **8**

Introduction . . . 9

2.1 Spécification des besoins . . . 9

2.1.1 Identification des acteurs . . . 9

2.1.2 Besoins fonctionnels par acteur . . . 10

2.1.3 Besoins non fonctionnels . . . 11

2.2 Diagramme de cas d'utilisation global . . . 12

2.3 Diagramme de classes global . . . 14

2.4 Structure et découpage du projet . . . 16

2.4.1 Product Backlog global . . . 16

2.4.2 Planification des releases 17
2.4.3 Planification des sprints 17
Conclusion 18

Conclusion générale et perspectives **19**

Webographie **20**

Table des figures

1.1 Cycle de vie SCRUM 6
2.1 Diagramme de cas d'utilisation — périmètre étayé par le dépôt 13
2.2 Diagramme de classes global — principaux modèles Prisma de la plateforme 15

Liste des tableaux

2.1 Acteurs de la plateforme et périmètre associé 9
2.2 Backlog produit proposé à partir des besoins et des écarts observés 16
2.3 Planification indicative des releases 17
2.4 Planification indicative des sprints 18

Liste des abréviations

API	Application Programming Interface
DTO	Data Transfer Object
JWT	JSON Web Token
ORM	Object-Relational Mapping
PFE	Projet de fin d'études
RPE	Rate of Perceived Exertion
UML	Unified Modeling Language

Introduction générale

Les applications numériques consacrées à l'activité physique regroupent aujourd'hui des fonctions de planification, de suivi et d'accompagnement. La plateforme étudiée réunit ces usages dans une application web et y associe des programmes sportifs ainsi qu'un espace de commerce. L'étendue de cette interface rend nécessaire une lecture précise des services effectivement disponibles : les pages visibles ne suffisent pas à établir qu'un parcours est persisté ou pris en charge par l'API.

Ce rapport s'appuie sur les fichiers présents dans le dépôt : application Next.js, API NestJS, schéma Prisma, contrôleurs, services, migrations et diagrammes. Il distingue les fonctionnalités reliées à des routes de celles qui sont seulement amorcées ou présentées par l'interface. Les informations d'entreprise, les résultats d'essais et le déroulement réel du projet ne figurant pas dans les sources, ils ne sont pas supposés.

Le premier chapitre situe le projet et expose la démarche de travail décrite dans le rapport. Le deuxième formalise les besoins, les acteurs, les diagrammes globaux et le découpage fonctionnel. La conclusion synthétise les constats et les suites à documenter.

Chapitre 1

Cadre général du projet

Plan

Introduction 3
1.1 Contexte général 3
1.2 Présentation du projet 3
1.2.1 Cadre général 3
1.2.2 Description du projet 3
1.2.3 Étude de l'existant 4
1.2.4 Problématique 4
1.2.5 Solution proposée 4
1.3 Présentation de la méthodologie de travail 5
1.3.1 Formalisme de modélisation 5
1.3.2 Méthodologie adoptée 5
Conclusion 7

Chapitre 1. Cadre général du projet

Introduction

Ce chapitre situe la plateforme Fitness dans son contexte, précise le périmètre observé dans le dépôt et présente les choix d'organisation et de modélisation retenus. Les constats décrivent l'implémentation disponible au moment de l'analyse ; les informations institutionnelles absentes du dépôt sont signalées comme telles.

1.1 Contexte général

Les outils web et mobiles occupent une place croissante dans l'organisation de l'entraînement personnel. La conception examinée répond à un besoin de regroupement des fonctions de préparation des séances, de suivi des résultats et de consultation de contenus sportifs, avec un espace complémentaire de vente d'articles. Le dépôt matérialise ce périmètre au moyen d'une interface web, d'une API et d'une base relationnelle.

1.2 Présentation du projet

1.2.1 Cadre général

Le projet est une plateforme web Fitness destinée à des utilisateurs qui souhaitent créer des plans d'entraînement, enregistrer des séances et suivre leur activité. Le système comprend également des programmes structurés sur plusieurs semaines, un catalogue de produits, un panier et un service backend de recommandations adaptatives fondées sur les données de performance.

1.2.2 Description du projet

L'interface est réalisée avec Next.js 14, React et TypeScript. Le backend est une API REST modulaire construite avec NestJS. La persistance s'appuie sur PostgreSQL et Prisma. L'authentification utilise des jetons JWT et le hachage des mots de passe par bcrypt. Le client consomme l'API au moyen d'Axios ; Zustand conserve l'état d'authentification côté interface. Recharts est installé pour les visualisations, et Stripe figure dans les dépendances et dans le service de commande.

Le schéma Prisma comprend les comptes et les données de profil et d'abonnement, les exercices et séances, les programmes, les journaux de performance ainsi que les modèles du catalogue et des commandes. Le backend répartit ces domaines entre des modules NestJS

dédiés. Les services accèdent directement à `PrismaService` ; aucune couche repository distincte n'apparaît dans le dépôt.

1.2.3 Étude de l'existant

L'analyse porte sur les sources, les pages Next.js, les contrôleurs et services NestJS, le schéma Prisma, les migrations, les fichiers de configuration et la documentation fonctionnelle présente dans le dépôt. L'état observé associe des parcours utilisables à des interfaces de démonstration et à des fonctionnalités préparées côté service mais non raccordées à des routes.

Les parcours soutenus par des contrôleurs comprennent l'inscription et la connexion, la consultation des exercices et programmes, la gestion des plans et séances, l'inscription à un programme et le suivi de sa progression, les statistiques de séances, les fonctions du panier et le service adaptatif. À l'inverse, le contrôleur de commandes ne déclare aucune route, bien que son service contienne des opérations de création de commande et d'intention de paiement Stripe. La page calendrier emploie des dates d'exemple ; l'écran cartographique indique explicitement que l'intégration de carte reste à venir. Certaines visualisations analytiques utilisent également des séries codées en dur.

1.2.4 Problématique

La difficulté principale mise en évidence par l'examen est l'écart entre l'étendue fonctionnelle affichée par l'interface et les fonctions réellement persistées et exposées par l'API. Les écrans de profil et d'abonnement ne disposent pas d'opérations backend de mise à jour ou de facturation ; les commandes ne sont pas accessibles via leur contrôleur ; et les routes d'écriture des contenus sont protégées par JWT sans contrôle effectif du rôle administrateur. La documentation doit donc distinguer clairement l'existant opérationnel, les éléments partiels et les évolutions à réaliser.

1.2.5 Solution proposée

La solution présente une architecture séparant l'interface, les services métier et la persistance. L'API centralise les opérations sur les données d'entraînement, le catalogue et le panier. Le moteur d'adaptation analyse les journaux de performance à l'aide de règles déterministes explicables ; il ne s'agit pas d'un fournisseur d'intelligence artificielle externe. Pour compléter le périmètre présenté par l'interface, les évolutions prioritaires consistent à raccorder les fonctions inachevées et à renforcer le contrôle des droits avant leur mise à disposition.

1.3 Présentation de la méthodologie de travail

Dans le volet méthodologique, nous présentons le formalisme de modélisation retenu pour décrire la plateforme, puis le cadre de développement indiqué dans le rapport initial. Le dépôt ne contient pas de journal de sprints permettant de confirmer le déroulement effectif de cette méthode.

1.3.1 Formalisme de modélisation

Pour la conception de notre plateforme Fitness, nous utilisons UML (Unified Modeling Language), un langage graphique de modélisation standard. UML complète les descriptions textuelles et facilite la compréhension de la structure du système ainsi que des interactions entre ses acteurs et ses fonctionnalités. Les diagrammes éditables du dépôt sont rédigés en PlantUML, qui transforme une description textuelle en diagramme [5].

Les diagrammes UML utilisés dans notre rapport sont les suivants :

— Le diagramme de cas d'utilisation représente les principales fonctionnalités de la plateforme accessibles aux acteurs, notamment l'authentification, le suivi des entraînements, les programmes et les services de la boutique.

— Le diagramme de classes présente les principales entités métier de la plateforme et les associations qui les relient, telles que les utilisateurs, les exercices, les programmes et les séances.

Les diagrammes de cas d'utilisation et de classes présentés au chapitre suivant illustrent ces deux vues complémentaires.

1.3.2 Méthodologie adoptée

À mesure qu'un projet logiciel devient complexe et regroupe plusieurs fonctionnalités interdépendantes, une méthodologie est nécessaire pour organiser le travail et coordonner les activités de développement. Elle définit les règles de réalisation, les rôles et responsabilités, l'enchaînement des activités, l'organisation des tâches ainsi que les livrables attendus.

Parmi les méthodologies agiles les plus utilisées, nous pouvons citer XP (Extreme Programming), Kanban, FDD (Feature Driven Development) et SCRUM.

Le rapport initial indique Scrum comme cadre retenu. Le Guide Scrum décrit ce cadre comme une approche itérative et incrémentale organisée autour d'une équipe, d'événements, d'artefacts et de sprints [6]. Il permet de répartir la réalisation en périodes courtes, de prioriser les exigences et de livrer progressivement des incréments fonctionnels. En l'absence de comptes rendus ou d'historique de projet, les releases et sprints décrits au chapitre suivant sont présentés comme une proposition de planification, non comme des itérations réalisées.

Ce cadre peut structurer une plateforme dont les fonctionnalités sont réparties entre plusieurs domaines, tels que l'entraînement, les programmes, le suivi des performances et la boutique. Il offre notamment les avantages suivants :

— une visibilité claire sur l'avancement du projet ;

— la possibilité de prioriser les fonctionnalités selon les besoins ;

— une adaptation aux changements des exigences ;

— le développement progressif des modules de la plateforme ;

— la division du projet en sprints facilement gérables ;

— la validation régulière des fonctionnalités développées.

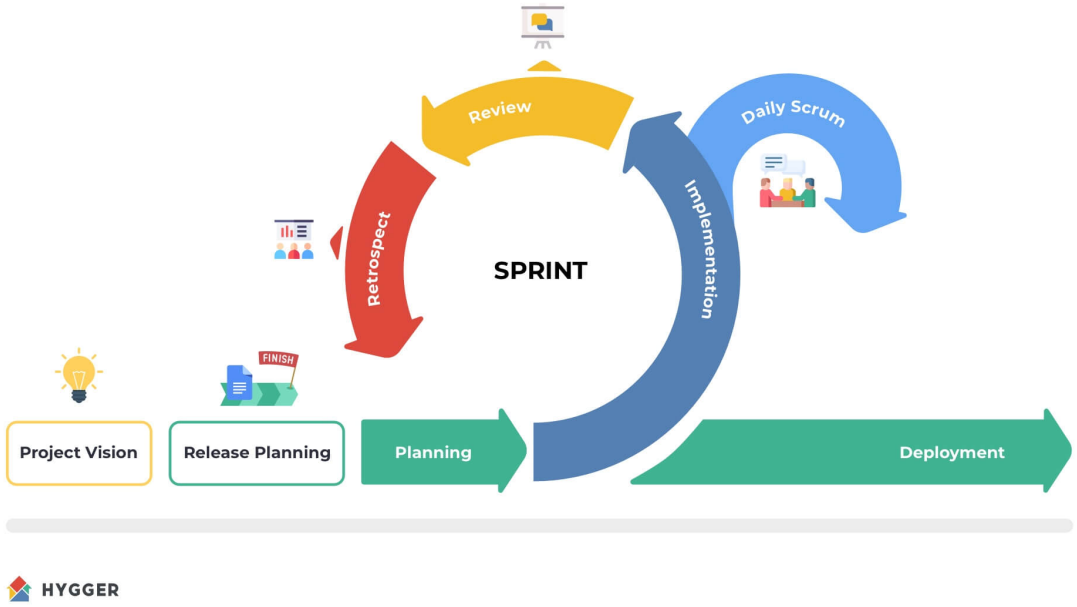


FIGURE 1.1 – Cycle de vie SCRUM

Le déroulement d'un sprint comprend les activités suivantes :

— **Sprint Planning :** l'équipe définit l'objectif du sprint et sélectionne les éléments prioritaires à réaliser.

— **Développement du sprint :** l'équipe conçoit, développe et vérifie les fonctionnalités retenues pendant la durée du sprint.

— **Daily Scrum :** un point quotidien permet de partager l'avancement et de coordonner le travail restant.

— **Sprint Review :** à la fin du sprint, l'équipe présente les fonctionnalités terminées et recueille les observations sur l'incrément produit.

— **Sprint Retrospective :** l'équipe examine le déroulement du sprint et identifie des améliorations à appliquer aux cycles suivants.

À l'issue du sprint, le **Product Increment** regroupe les fonctionnalités réalisées et intégrées. Il peut constituer une version potentiellement livrable de notre plateforme. La rétrospective permet à l'équipe d'améliorer progressivement son organisation et sa manière de travailler. Le chapitre suivant présente les besoins, les diagrammes globaux, puis le backlog, les releases et la planification des sprints de notre projet.

Conclusion

Ce chapitre a présenté le cadre logiciel, les technologies et les principaux écarts constatés entre les interfaces et les services disponibles. Cette distinction fonde l'analyse des besoins et la planification de compléments réalistes au chapitre suivant.

Chapitre 2

Analyse et spécification des besoins

Plan

Introduction 9
2.1 Spécification des besoins 9
2.1.1 Identification des acteurs 9
2.1.2 Besoins fonctionnels par acteur 10
2.1.3 Besoins non fonctionnels 11
2.2 Diagramme de cas d'utilisation global 12
2.3 Diagramme de classes global 14
2.4 Structure et découpage du projet 16
2.4.1 Product Backlog global 16
2.4.2 Planification des releases 17
2.4.3 Planification des sprints 17
Conclusion 18

Introduction

Ce chapitre formalise les acteurs et les exigences de la plateforme, puis présente les diagrammes UML et le découpage fonctionnel. Les besoins sont qualifiés selon leur état dans le dépôt : implémenté par une route/service, partiel ou uniquement représenté par l'interface, ou à compléter.

2.1 Spécification des besoins

2.1.1 Identification des acteurs

TABLE 2.1 – Acteurs de la plateforme et périmètre associé

Acteur	**Rôle dans le périmètre observé**
Visiteur	Crée un compte, se connecte et consulte les contenus publics de programmes et de produits.
Utilisateur authentifié	Gère ses plans et séances, suit des programmes, consulte ses statistiques, utilise le panier et les fonctions de suivi adaptatif.
Administrateur	Rôle prévu par l'attribut `User.role` et par l'interface d'administration. Le backend ne vérifie pas actuellement ce rôle dans les opérations d'écriture ; cet acteur représente donc une responsabilité cible plutôt qu'une autorisation correctement appliquée.
Moteur adaptatif	Composant interne qui calcule des indicateurs et recommandations à partir des performances. Il est fondé sur des règles déterministes et ne constitue pas un système externe d'IA.

Stripe est un système externe de paiement, et non un acteur de la plateforme. Le service contient une logique d'intention de paiement, mais le contrôleur de commandes ne l'expose pas.

2.1.2 Besoins fonctionnels par acteur

Les besoins fonctionnels de l'application sont définis selon les différents acteurs et les fonctionnalités attendues. L'état constaté correspond à l'état actuel de leur implémentation dans le projet.

Visiteur

Le visiteur doit pouvoir :

— S'inscrire et s'authentifier.

— Parcourir les programmes d'entraînement et le catalogue public des produits.

État constaté : Les routes d'authentification, des programmes et des produits sont présentes.

Utilisateur

L'utilisateur doit pouvoir :

— Consulter son identité et gérer son profil.

— Créer et gérer ses plans d'entraînement.

— Démarrer, terminer ou annuler une séance et consulter son historique.

État constaté : Les routes protégées sont présentes. La modification du profil est absente.

L'utilisateur doit également pouvoir :

— Suivre un programme d'entraînement.

— Consulter son inscription et mettre à jour l'avancement des séances du programme.

État constaté : Les routes correspondantes sont présentes et le contrôle par JWT est utilisé.

Par ailleurs, l'utilisateur doit pouvoir :

— Consulter ses statistiques.

— Enregistrer ses performances.

— Obtenir des recommandations adaptatives.

État constaté : Les statistiques de base et le service adaptatif sont présents. Certaines pages affichent toutefois des données d'exemple.

Concernant la marketplace, l'utilisateur doit pouvoir :

— Parcourir les produits.

— Ajouter des articles au panier.

— Modifier les quantités et vider le panier.

État constaté : Le catalogue et les routes du panier sont présents.

Enfin, l'utilisateur doit pouvoir :

— Planifier ses séances dans le calendrier.

— Gérer son abonnement.

— Effectuer une commande payée.

État constaté : L'interface et le calendrier sont partiels. Il n'existe pas d'API de programmation ou d'abonnement, et le contrôleur des commandes est vide.

Administrateur

L'administrateur doit pouvoir :

— Gérer les utilisateurs.

— Gérer les exercices et les programmes.

— Gérer les produits et les commandes.

État constaté : Les opérations CRUD de certains contenus sont disponibles derrière une authentification JWT. Cependant, la gestion des utilisateurs est absente, aucun contrôle de rôle n'est implémenté et les commandes ne disposent pas de routes dédiées.

2.1.3 Besoins non fonctionnels

— **Sécurité :** authentification JWT, mots de passe hachés avec bcrypt, validation globale des entrées via `ValidationPipe`. Le contrôle de rôle administrateur et les routes d'abonnement/commande restent à compléter.

— **Intégrité :** PostgreSQL et contraintes/relations assurent la persistance relationnelle ; les DTO valident les données des routes concernées.

— **Maintenabilité :** modules NestJS séparés par domaine, composants et pages Next.js, types TypeScript et schéma centralisé.

— **Interopérabilité :** échanges entre frontend et backend par API REST ; la configuration CORS observée autorise actuellement l'origine de développement.

— **Ergonomie et adaptation :** interface web responsive visée ; l'expérience doit distinguer les données réellement disponibles des éléments de démonstration.

— **Confidentialité :** les données d'entraînement étant rattachées aux utilisateurs, les opérations doivent rester limitées au propriétaire ; les contrôles d'accès doivent être vérifiés pour chaque ressource.

2.2 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global présente les principaux parcours fonctionnels visés pour l'application Fitness, du point de vue des utilisateurs et de l'administrateur. Il inclut les services externes représentés dans la vue fonctionnelle. Cette figure exprime le périmètre fonctionnel attendu ; elle ne signifie pas que chaque parcours est déjà exposé par une route backend. L'état d'implémentation et les limites relevées dans le dépôt sont décrits dans les sections correspondantes. La figure 2.1 complète l'état détaillé des besoins du tableau **??**.

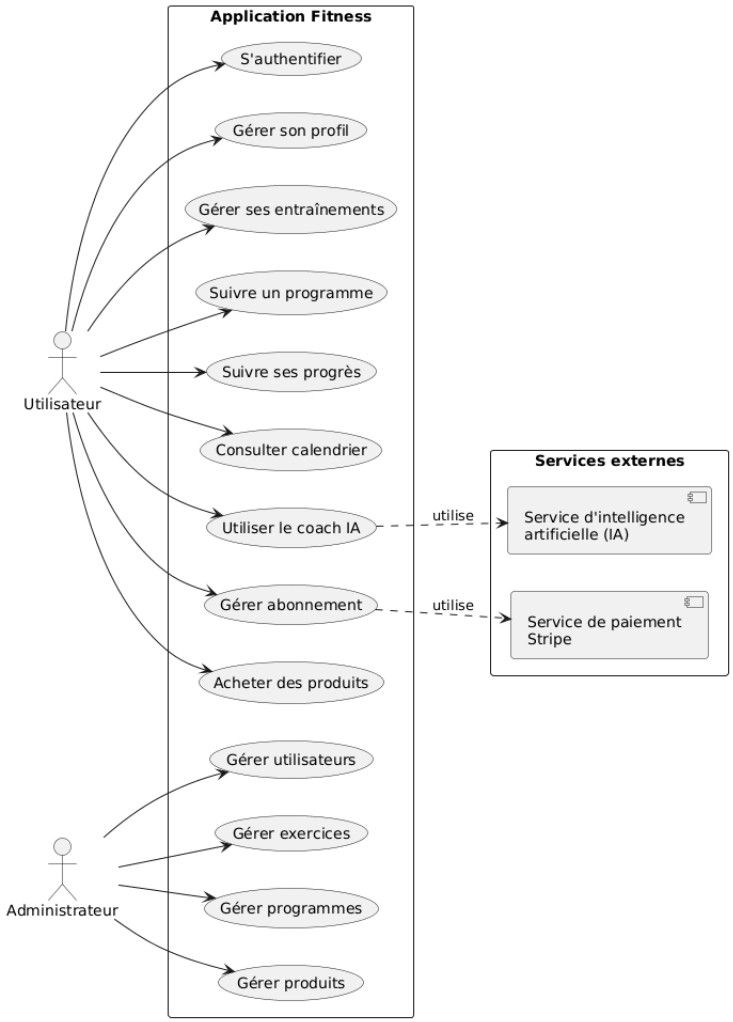


FIGURE 2.1 – Diagramme de cas d'utilisation — périmètre étayé par le dépôt

2.3 Diagramme de classes global

Le diagramme de classes global présente une vue métier d'ensemble des utilisateurs, programmes, séances, performances, abonnements, paiements et produits. Il rassemble les classes et relations de la conception fonctionnelle illustrée par la figure 2.2 ; il ne constitue pas une transcription du schéma Prisma. Les différences entre ce modèle métier et la persistance réellement déclarée sont précisées ci-dessous.

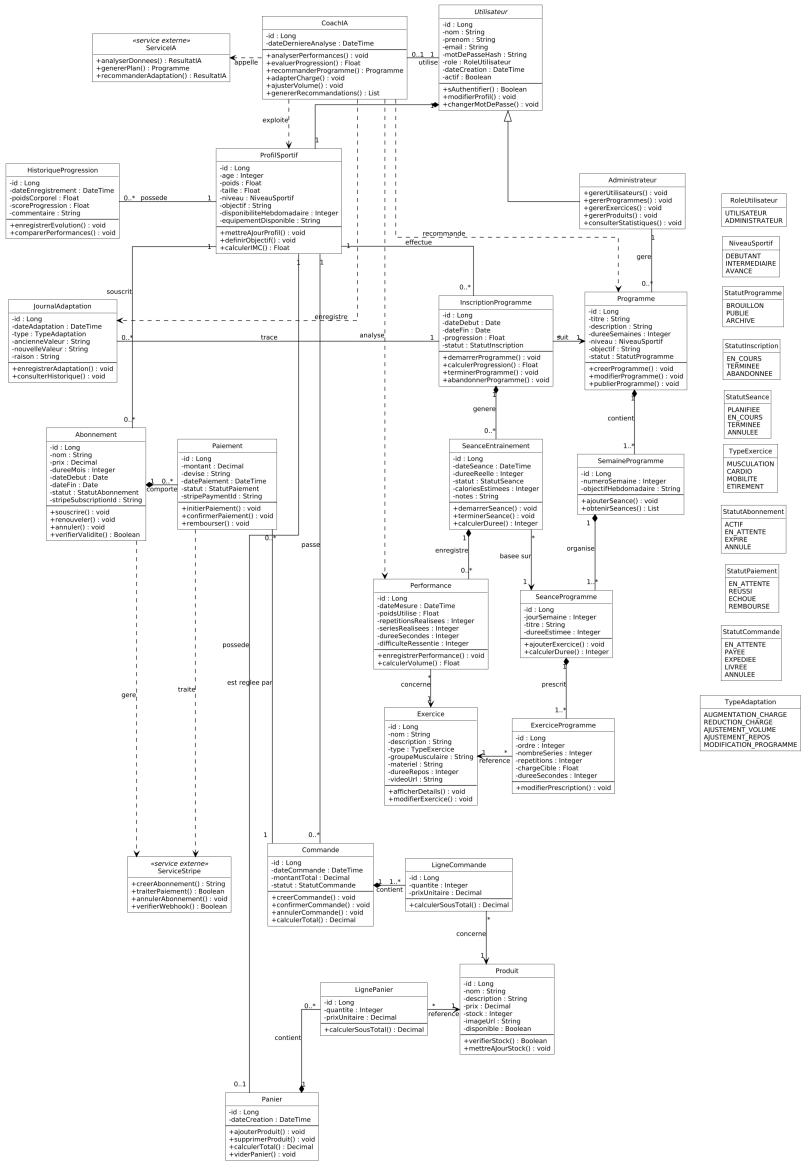


FIGURE 2.2 – Diagramme de classes global — principaux modèles Prisma de la plateforme

Dans l'implémentation Prisma, les informations de profil et d'abonnement (taille, poids, objectif, formule et état) sont portées par `User` ; les modèles distincts `ProfilSportif` et `Abonnement` du diagramme n'existent pas. Les données de paiement sont portées par `Order`, sans modèle `Paiement` séparé. L'adaptation est journalisée par `AdaptationLog` ; le coaching est assuré par une logique backend et aucun modèle `CoachIA` ni service externe d'IA n'est déclaré. Les modèles de commande et de paiement ne disposent pas non plus de routes backend exposées à ce jour.

2.4 Structure et découpage du projet

Le découpage fonctionnel suit les modules présents dans l'API et les espaces de l'interface : authentification, utilisateurs, exercices, plans et séances, programmes, adaptation, catalogue, panier et commandes. Les priorités ci-dessous combinent l'état observé et l'ordre proposé pour terminer les parcours. Les durées sont des estimations indicatives en jours ouvrés, et ne proviennent pas d'un outil de suivi de projet.

2.4.1 Product Backlog global

TABLE 2.2 – Backlog produit proposé à partir des besoins et des écarts observés

ID	Fonctionnalité	ID	User Story	Priorité	Durée
F01	Authentification et profil	PB01	En tant que visiteur, je veux créer un compte et me connecter afin d'accéder à mon espace.	P1	2 j
F01	Authentification et profil	PB02	En tant qu'utilisateur, je veux consulter mes informations de compte et mettre à jour mon profil.	P1	1,5 j
F02	Exercices et entraînements	PB03	En tant qu'utilisateur, je veux consulter et gérer les exercices proposés.	P1	2,5 j
F02	Exercices et entraînements	PB04	En tant qu'utilisateur, je veux créer et gérer mes plans d'entraînement.	P1	2,5 j
F02	Exercices et entraînements	PB05	En tant qu'utilisateur, je veux démarrer, terminer, annuler et retrouver mes séances.	P1	2,5 j
F02	Exercices et entraînements	PB06	En tant qu'utilisateur, je veux enregistrer mes séries, charges, répétitions et RPE.	P1	2,5 j
F02	Exercices et entraînements	PB07	En tant qu'utilisateur, je veux consulter mes statistiques issues de mes séances.	P1	1,5 j
F03	Programmes sportifs	PB08	En tant qu'utilisateur, je veux parcourir des programmes et consulter leur contenu.	P1	1,5 j
F03	Programmes sportifs	PB09	En tant qu'utilisateur, je veux m'inscrire à un programme et suivre son avancement.	P1	2,5 j
F04	Coaching adaptatif	PB10	En tant qu'utilisateur, je veux obtenir des conseils adaptés à mes performances récentes.	P2	4 j
F05	Calendrier sportif	PB11	En tant qu'utilisateur, je veux consulter et planifier mon calendrier d'entraînement.	P2	2,5 j
F06	Boutique et commandes	PB12	En tant qu'utilisateur, je veux consulter les produits et leurs détails.	P2	1,5 j
F06	Boutique et commandes	PB13	En tant qu'utilisateur, je veux ajouter, modifier et retirer des articles de mon panier.	P2	2,5 j

ID	Fonctionnalité	ID	User Story	Priorité	Durée
F06	Boutique et commandes	PB14	En tant qu'utilisateur, je veux créer une commande et payer avec Stripe.	P2	4 j
F06	Boutique et commandes	PB15	En tant qu'utilisateur, je veux consulter mes commandes et leur état.	P2	2,5 j
F07	Abonnements	PB16	En tant qu'abonné, je veux choisir, payer et gérer une formule Free, Pro ou Elite.	P2	4 j
F08	Administration	PB17	En tant qu'administrateur, je veux administrer les produits, programmes et exercices avec des droits vérifiés.	P1	4 j
F08	Administration	PB18	En tant qu'administrateur, je veux gérer les utilisateurs et les commandes.	P3	4 j
F09	Activités extérieures	PB19	En tant qu'utilisateur pratiquant une activité extérieure, je veux visualiser mes parcours sur une carte.	P3	4 j

P1 désigne une priorité haute pour la cohérence et la sécurité du socle ; P2 désigne une capacité importante à compléter après le socle ; P3 correspond à une évolution secondaire. Les durées estimées couvrent l'analyse, l'implémentation et la validation d'une user story dans un contexte d'équipe ; elles devront être réévaluées selon les critères d'acceptation et la vélocité réels.

2.4.2 Planification des releases

TABLE 2.3 – Planification indicative des releases

Release	Objectif	Contenu proposé	État de référence
R1	Socle d'entraînement	Authentification, exercices, plans, séances, historique et statistiques réelles.	Majoritairement présent ; profil éditable à compléter.
R2	Programmes et adaptation	Catalogue des programmes, inscription, progression, journal de performance et conseils adaptatifs.	API et logique principales présentes ; raccordement UI à vérifier.
R3	Commerce	Catalogue, panier, création de commande, paiement Stripe, suivi de commande et administration sécurisée.	Catalogue et panier présents ; commande/paiement non exposés par API.
R4	Services complémentaires	Cycle d'abonnement, calendrier persistant, administration des comptes et carte d'activités.	Partiel ou absent selon fonctionnalité.

Ces releases constituent une proposition de livraison fondée sur l'état du code et ne correspondent pas à des versions déjà publiées documentées dans le dépôt.

2.4.3 Planification des sprints

Table 2.4 – Planification indicative des sprints

Sprint	But	Éléments proposés	Livrable
S1	Fiabiliser le socle	Valider inscription/connexion, lecture du compte, CRUD des exercices et plans ; ajouter l'édition du profil et les tests d'accès par propriétaire.	R1 cohérente.
S2	Terminer le suivi sportif	Valider le cycle de vie des séances, compléter l'enregistrement des séries et remplacer les statistiques codées en dur par des données API.	Suivi basé sur données réelles.
S3	Consolider les programmes	Vérifier consultation, inscription et progression ; raccorder de manière fiable le panneau d'adaptation et distinguer la logique déterministe d'une IA externe.	R2 intégrée.
S4	Sécuriser et terminer le commerce	Ajouter contrôle de rôle, routes d'orders, paiement Stripe, traitement confirmé, consultation des commandes et gestion cohérente du stock.	Parcours d'achat complet.
S5	Compléter les services premium	Ajouter persistance et API du calendrier, cycle de facturation des abonnements et intégration cartographique si ces fonctions restent dans le périmètre.	R4 selon arbitrage.

L'ordre et les livrables sont une planification indicative déduite des lacunes ; aucune durée ni vélocité n'est inventée en l'absence de calendrier de projet et de données d'équipe.

Conclusion

L'analyse distingue les fonctions soutenues par l'API des interfaces de démonstration et des services non exposés. Les diagrammes décrivent le périmètre fonctionnel et un échantillon des entités réellement définies, tandis que le backlog traduit les écarts relevés en un ordre de réalisation cohérent. Cette base peut être enrichie par les informations institutionnelles et le suivi réel de l'équipe.

Conclusion générale et perspectives

Ce rapport a étudié une plateforme web de gestion de l'entraînement et de programmes sportifs, complétée par un espace de commerce. L'analyse du dépôt a permis de décrire son architecture Next.js, NestJS et PostgreSQL/Prisma, ainsi que les principaux modèles et parcours métier. Elle a également mis en évidence la nécessité de distinguer les pages de démonstration, les services présents et les opérations réellement exposées par l'API.

Les suites identifiées concernent le raccordement des parcours incomplets, la vérification des droits d'administration et la production d'éléments de validation reproductibles. Pour rendre le rapport pleinement représentatif d'un PFE, il reste à intégrer les informations institutionnelles et d'encadrement, la chronologie réelle du projet, les décisions méthodologiques effectivement suivies, les résultats des tests réalisés et, le cas échéant, les preuves de déploiement. Ces informations ne peuvent être établies à partir du dépôt seul.

Webographie

[1] Vercel, *Next.js Documentation*, https://nextjs.org/docs, consulté le 11 octobre 2026.

[2] NestJS, *Documentation officielle*, https://docs.nestjs.com/, consulté le 11 octobre 2026.

[3] Prisma, *Upgrade to Prisma ORM v5*, https://www.prisma.io/docs/guides/upgrade-prisma-orm/v5, consulté le 11 octobre 2026.

[4] PostgreSQL Global Development Group, *PostgreSQL Documentation*, https://www.postgresql.org/docs/current/, consulté le 11 octobre 2026.

[5] PlantUML, *Language Reference Guide*, https://plantuml.com/, consulté le 11 octobre 2026.

[6] Ken Schwaber et Jeff Sutherland, *The Scrum Guide*, novembre 2020, https://scrumguides.org/scrum-guide.html, consulté le 11 octobre 2026.